IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 6 May 2024, accepted 27 May 2024, date of publication 4 June 2024, date of current version 13 June 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3409433

RESEARCH ARTICLE

DRIFTNET-EnVACK: Adaptive Drift Detection in Cloud Data Streams With Ensemble Variational Auto-Encoder Featuring Contextual Network

TAJWAR MEHMOOD[1], SEEMAB LATIF[1], (Senior Member, IEEE), RABIA LATIF[2], HAMMAD MAJEED[3], AND ASAD WAQAR MALIK[1]

[1]School of Electrical Engineering and Computer Science (SEECS), National University of Sciences and Technology (NUST), Islamabad 44000, Pakistan
[2]College of Computer and Information Sciences (CCIS), Prince Sultan University, Riyadh 12435, Saudi Arabia
[3]Computer Science Department, FAST National University of Computer and Emerging Sciences, Islamabad 44000, Pakistan

Corresponding author: Seemab Latif (seemab.latif@seecs.edu.pk)

This work was supported in part by Prince Sultan University, and in part by SEECS CPInS Lab.

ABSTRACT The adoption of cloud computing has been increasingly common across several industries in recent years and offers unparalleled flexibility and scalability in managing computational resources. However, the increasing reliance on cloud infrastructure also raises concerns regarding the monitoring and mitigation of drift, which pertains to unforeseen modifications inside the cloud environment. The aforementioned modifications possess the capacity to induce operational disruptions and can affect cloud provider reputation. This paper introduces a novel methodology for a proactive drift detection approach, referred to as the Ensemble of Variational Auto-encoder featuring Sequential Contextual Network Extended Kolmogorov-Smirnov Test (EnVACK) a Multi-Mode Variational Auto-encoder with Kolmogorov-Smirnov. It analyzes the data distribution using a non-parametric deep learning approach to create the concept drift map to address sudden and gradual drift challenges in the non-Gaussian cloud domain. This map gives a drift overview at each resource and combined level. This has shown a 96% f-score in the cloud domain an improvement of 4% when evaluated against benchmark studies executed in different domains. The present methodology has been specifically devised to warn and predict the occurrence of drift in synthetic and real-world non-Gaussian cloud usage traces.

INDEX TERMS CPU usage, concept drift, drift map, drift detection, KS, memory usage, non-Gaussian distribution.

I. INTRODUCTION

The advent of cloud computing has brought a massive transformation since it allows users to conveniently access computer resources with flexibility and ease [1], [2], [3]. Organizations globally depend on cloud service providers to effectively oversee their workloads. The aforementioned features encompass on-demand self-service, ubiquitous network connectivity, resource pooling, quick elasticity, and measurable services, which collectively guarantee that customers are solely billed for the resources they utilize [4], [5], [6].

Nevertheless, the inherent dynamic character of cloud systems renders them susceptible to a process commonly referred to as ''cloud drift''. Cloud drift is a phenomenon characterized by inadvertent and gradual alterations that transpire within the cloud infrastructure. These modifications have the potential to result in diminished performance, cost, and energy consumption [7], [8], [9]. According to a study conducted by Google, it has been found that the

The associate editor coordinating the review of this manuscript and approving it for publication was Alberto Cano.

energy consumption of idle resources within data centres accounts for about 50% of the total energy consumption [10]. Additionally, the study reveals that energy consumption increases proportionally with resource usage [11]. Thus, the acknowledgement and mitigation of cloud drift are crucial factors in safeguarding the integrity of applications hosted on cloud systems and cloud providers.

This paper offers valuable contributions to the ever-evolving field of cloud computing. One of the most significant advancements relates to the integration of proactive cloud drift detection. Through the use of our methodology, cloud administrators are provided with the capacity to proactively identify and acknowledge emerging cloud drift patterns. The implementation of a proactive strategy would significantly reduce the likelihood of operational disruptions, thereby guaranteeing the reliability of cloud-based services. Furthermore, our technique promotes the efficient allocation and utilization of resources by helping the resource usage predictor. Models are frequently retrained to maintain performance, especially when data shifts occur. The effective handling of cloud drift contributes to the optimal distribution of resources within organizations, leading to cost advantages and improved system functionality. This cost-effective technique aligns seamlessly with the aims of organizations seeking to maximize the benefits of cloud computing. Cloud usage traces frequently have extremely skewed distributions and variations, conventional statistical techniques are inadequate. To solve this, our major goal is to create a system that efficiently detects and mitigates drift in cloud usage traces by combining statistical and deep learning techniques, utilizing the advantages of both approaches to handle the complexity of the data distribution. A Multi-Model Variational Auto-encoder (VAE) and Kolmogorov-Smirnov (KS) thresholding are used to detect and reduce cloud drift patterns. Auto-encoder research has made significant progress in recent times and has gained widespread use in the fields of deep learning and computer vision due to its ability to encode intricate input data. Auto-encoders, originally used for dimensionality reduction, now include convolutional [12], feature extraction [13], fault detection [14], anomaly detection [15] and denoising Auto-encoders [16], [17]. Variational Auto-encoders (VAEs) have garnered significant interest in the field of unsupervised learning [18], [19] and probabilistic generative modelling [20], [21], [22]. VAEs are commonly used for sampling and data dispersion interpretation. In recent times, there have been notable advancements in the performance of generative models. Data drift can be detected through dynamic datasets. These models can discover and modify data disparities, making them useful for managing enormous datasets. Their amazing capacity to capture non-linear correlations and intricate feature interactions gives them an advantage over typical statistical methodologies. They capture complex dataset patterns, adapt to shifting data distributions, and handle dynamic datasets. The capacity to detect small data changes is essential for real-world machine learning algorithm reliability and efficacy. This approach facilitates proactive monitoring capabilities for cloud managers, enabling them to identify, detect, and fix possible issues arising from cloud drift.

The paper's significant contributions are as follows:

- We propose a drift detector that combines the strengths of statistical and deep learning to achieve improved results over previously introduced Dynamic Cloud Resource Utilization Dataset (DCRUD) .
- An overview of the issues associated with drifts in cloud usage trace, as well as potential future approaches and methods for mitigating these challenges.
- We study the effects of tiny alterations in data that may have been overlooked by individual methods by considering both gradual and sudden drift at task and window levels.
- We demonstrate rigid conceptual assurances for false positive and false negative rates in our change detectors.
- We show that our successful management of non-linear data relationships in real-world cloud traces can be achieved by implementing a concept drift map, which encompasses both resource-specific and aggregate resilient approaches for continual surveillance.

This document is structured in the following manner to facilitate understanding of our research: Section II presents a comprehensive examination of the existing body of literature on cloud drift detection, encompassing various research studies and methodologies. The originality and innovation of our proposed model are established in this section. In Section III, the methodology is presented, which outlines the implementation of a Multi-Model VAE with contextual network and KS thresholding to detect drift in cloud usage traces. The architecture, algorithms, and approaches of our model are elucidated in this document. Section IV provides a comprehensive account of the datasets, experiments, and outcomes conducted as part of our Experimental Evaluation. In Section V, the Discussion and Implications section, an in-depth analysis is presented of the obtained findings and their relevance in the context of cloud computing. The final section of this study presents the conclusions drawn from our findings and highlights prospective avenues for future research. Specifically, it emphasizes the significance of enhancing cloud drift detection and prevention techniques.

II. LITERATURE REVIEW

Numerous rigorous academic studies have been undertaken on the topic of detecting and compensating for concept drift. Considerable endeavours have been conducted by scholars in the disciplines of statistics and deep learning to investigate the phenomenon of concept drift detection. These programs aim to maintain the accuracy and reliability of models in changing environments. The goal is to handle the model trained on past data to exhibit comparable performance on novel data by checking alteration in distribution resulting in reduced accuracy. Concept drift changes patterns that require optimum model updating according to contexts.

Concept change detection methods often have low sensitivity to small changes, and high false positive rates resulting in high computing costs by unnecessary complex model adaptation [23]. Thus, concept drift detection refers to the identification of changes in the underlying data distribution, which aids in determining the appropriate timing for model updates [24]. When modifying learning models to incorporate modifications, the stability-plasticity dilemma entails the delicate equilibrium between stability, which entails the preservation of valuable knowledge, and plasticity, which requires the ability to adapt to novel information [25]. Thus, the literature review covers the existing drift detector in the context of stability-plasticity. The concept drift detection method is commonly classified into five primary groups in terms of their detection strategy within the literature.

Sequential drift detection utilizes a single time series for analysis. ADWIN [26], SEED [27], Sequential Drift (SeqDrift) 1 and 2 [23] and HDDMA/HDDMW [28] employ continuous values while it is possible for these detectors to disregard correlation and prioritize sequential analysis. Thus, they are not capable of effectively analyzing regression data. They also suffer from unnecessary model updation. In **performance or learner-based** approaches, the output of the base learner is utilized as a point of [29]. Popular drift detectors used in the field of machine learning include DDM [30] and EDDM [31], which are designed to identify drift based on learner output. Nishida and Yamauchi introduces the Statistical Test of Equal Proportions (STEPD) [32] for identifying concept drift. Ross et al. propose the use of EWMzA algorithms for Concept Drift Detection (ECDD) [33] Additionally, Barros et al. present the Reactive Drift Detection Method (RDDM) for the same purpose [34]. The task at hand is to detect and characterize instances of category output drift. DDM was further integrated into complicated frameworks to address notion drift. Meta-RRKOS-ELM-DDM, a modified DDM for time series forecasting, was introduced to address idea drift and reduce learning time [35]. The extent of data required for effective adaptation to dynamic environments is not explicitly disclosed, thereby making categorical output detectors more proficient in detecting drift rather than comprehending it. The provision of prompt feedback is advantageous; nevertheless, it is not feasible to comprehensively address the intricacies of data distribution. To interpret the drift, **contextually** based detectors are utilized, together with data and model assistance. Information about the context is taken from the model, such as checking the development of leveraging past drift patterns [36], spiking neural network [37], graph metrics [38], [39], or VAE [40] are utilized in this process. They use the model to acquire a deep understanding of the alteration of the distribution, enabling a thorough understanding of the occurrence and, particularly, a deeper understanding of the association between the attributes impacted by this drift. Spiking neural network [37] and VAE [40] lead more toward the use of deep learning for drift detection. In [40], they focused on using VAEs as drift detectors as they effectively reflect the underlying distribution of data. They highlighted its importance over other statistical techniques due to the incorporation of probabilistic modelling into their architecture to capture the underlying uncertainty and intricate structures present in the data.

Data distribution-based methods evaluated window distribution similarities, which might lead to false positives. Using a test statistic, these methods assess old and new data similarity, with the null hypothesis implying no idea drift and rejection requiring action [41]. These use data distribution analysis to detect regression and multivariate data changes. These methods collect drift data via attribute correlation. The amplitude, occurrence rate, and duration of drift are shown, helping refine conceptual frameworks. Data distribution-based approaches can detect idea drift in variables unrelated to the learner's output, causing unnecessary machine learning model changes. The Concept Drift Map approach analyzes data distribution changes to detect drift. Idea drift can be detected, however this approach may also identify factors unrelated to the learner's output, resulting in unnecessary model updates. The study [42] suggests using a concept drift map to better understand attribute drift. Concept drift maps assess drift by correlating numerous parameters. Webb et al. introduced a probability-based idea drift map using Naive Bayes [42]. Discrete values make this process difficult. Thus, continuous values must be binned before determining distance. Translation from a continuous process to a categorical conclusion is inefficient. Transcoding destroys vital data. Thus, discretizing the previous dataset may not work for the present distribution. Yu and Webb [43] propose an adaptive concept drift map for continuous values to overcome strong distribution-type assumptions. Comparing the average mean and maximum values of two Gaussian distributions revealed drift. The works discussed here dive deeper into the field of cloud trace data analysis, highlighting the need to address data representation challenges. Unlike previous approaches [42] that relied exclusively on discrete values, these researchers acknowledge that this discretization can result in the loss of valuable information. Another restriction comes when the data is assumed to have a Gaussian distribution [43], which is unsuitable for cloud usage traces due to their frequently extremely skewed distributions. The fundamental goal of these investigations is to pioneer a novel methodology that bridges the gap between continuous values and non-parametric methods, thereby significantly improving the identification and mitigation of drift in cloud usage traces. This novel technique aims to address the problem by leveraging the strengths of both continuous and non-parametric methodologies. Box plot visualization in Figure 1 helps identify and comprehend the presence of large variations from the norm because it gives a clear picture of the number of drift episodes in a dataset by highlighting extreme values. Instead, a violin plot in Figure 1 provides a visual understanding of the data shape and density, revealing

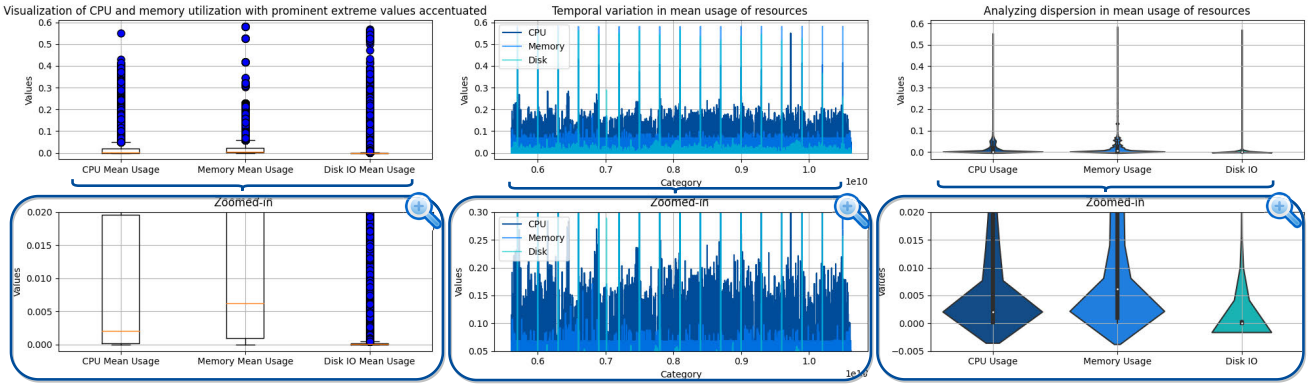

FIGURE 1. Exploration of temporal resource utilization trend: Uncovering data asymmetry and highlighting peculiar extreme values.

TABLE 1. Comparison of concept drift detection categories.

Category	Multi- variate time series	Regression	Information about data adaption	Unnecessary Model adaption	Model Approach	Variables correlation
Sequential analysis	No	Yes	Low	No	Proactive	No
Performance output	Yes	No	No	Low	Reactive	No
Data distribution	Yes	Yes	High	High	Proactive	Yes
Contextual	Yes	Yes	High	High	Proactive	Yes

not only the presence of drift but also the asymmetry and concentration of values within the distribution. This makes it particularly useful for illustrating the highly skewed distribution of CPU and memory usage. Together, these visualization methods with the time series plot of CPU, memory, and disk IO in Figure 1 offer insightful information on the frequency of drift events and the characteristics of resource usage data distribution, facilitating the thorough study of drift and the identification of changes in data patterns over time.

Multiple hypotheses-based detectors, to make a detection, used a mixture of several different detection methods. This approach allows us to hybrid data distribution with a contextual drift detector. These methods are mentioned in a survey paper by Lu et.al. [24] where they are combined either concurrently or hierarchically. Comprehensive and adaptive concept drift detection approaches are emerging. In deep learning concept drift detection, resilient methods to handle changing data distributions are becoming more popular. Traditional ensemble learning and sliding window approaches are well-studied. However, approaches like OS-ELM [44] and Dynamic Extreme Learning Machine [45] stress data flexibility and measure the learner performance degradation. The Metacognitive Online Sequential Extreme Learning Machine [46] also addresses domain knowledge, imbalanced datasets, and delayed versus quick concept transitions. Deep learning models that need dynamic real-world environments fuel this trend. The distribution of real-world data will always differ from model training. The [47] binary classifier detects comparable changes in image distribution when detecting out-of-distribution. The article [48] discusses the use of a supervised learning approach for detecting drift in the cloud. However, the use of LSTMDD still leaves a gap for unsupervised learning.

In conclusion, the comparison of concept drift detectors summarised in Table 1 stands as a pivotal endeavor within the rapidly evolving sphere of deep learning. This is particularly pertinent in scenarios where data undergoes continual changes. Within this pantheon of detectors lie distinctive advantages and nuanced challenges. Sequential detectors bestow the gift of real-time adaptability, affording immediate response to evolving data dynamics. Thus, they empower models to swiftly adapt, ensuring continuous reliability for a single series of data. Meanwhile, performance-based detectors operate with acumen, acting reactively based on model performance metrics. This reactive approach remains a crucial safety net to intervene when other methods may falter. Data distribution detectors, on the other hand, serve as sentinels of early drift detection, enabling proactive intervention. They facilitate early alerts using information about data adaption, preventing significant model degradation. Finally, Contextual detectors offer a deeper understanding of context-dependent drift, allowing for precise adjustments. It brings an additional layer of sophistication by contextualizing data transformations within their surrounding milieu. Highlighting the advantages of these systems, it is imperative to underscore that they enable models to maintain peak performance even in the face of dynamic data shifts. In practical terms, the selection of a drift detector ought to be a judicious process for the cloud domain. There is a need for a novel approach artfully guided by the specific exigencies and unique characteristics of the cloud domain. As a result,

our contribution to drift detection is innovative and integrates hypothesis-based detectors the combo of contextual and data distribution detector natures. Data distribution-based drift detectors don't have a continuous quantitative measure for the non-parametric distribution of cloud usage to get a detailed description of the nature and form of drifts. Literature shows a gap for a concept drift mapping technique specifically designed for cloud non-Gaussian continuous resource usage values.

This novel approach successfully handles the issue of adapting variations in cloud trace resources. To minimize information loss and fully understand the complex variations of drift correlations within the context, it is crucial to emphasize the continuous character of data streams. To comprehend the progressive nature of drift patterns more fully, it is crucial to maintain contextual awareness. A proactive strategy is necessary in the context of cloud settings because a reactive one falls short of meeting the demands efficiently. To address the issues of drift detection within cloud computing, our proposed proactive distribution and contextual-based detectors constitute an encouraging step forward.

TABLE 2. Data characteristics.

Datasets	Google [49]	Sine [50], [51]
Number of Instance	10,48575	1,000,000
Number of Attribute	17	24
Drift Type	Abrupt & Gradual	Abrupt
Dataset Type	Real	Artificial

III. METHODOLOGY

A. DATASET

In this experiment, we have selected three datasets for evaluation: DCRUD , Hyper, and Sine. Both Hyper and Sine are synthetic drift datasets as benchmarks to test model performance. The DCRUD dataset represents time series data to represent the cloud computing system.

B. DYNAMIC CLOUD RESOURCE UTILIZATION DATASET (DCRUD)

The DCRUD offers a detailed insight into the workload dynamics of one of the world's leading technology giants. The dataset provides a comprehensive overview of how tasks utilize computer resources within our clusters. In this dataset drift exists, meaning that the underlying distribution changes over time. This highlights the importance of detecting and adapting to concept drift to maintain the accuracy and performance of the models. Furthermore, it is specified that all attributes in these datasets are continuous. To ensure fair comparisons and consistency, it is mentioned that the values of all parameters in the datasets are normalized, ranging from 0 to 1. Normalization is a common pre-processing technique that scales the data to a standardized range, making it easier to compare and analyze the values across different attributes. The dataset contains metrics such as CPU utilization, memory utilization, and network traffic. The data is collected at regular intervals of five minutes, resulting in a total of 10,48575 samples. Datasets characteristics are mentioned in Table 2. This data presents a granular and detailed look into how tasks function and consume resources within our computer clusters.

This provides a comprehensive view of Google's extensive computing infrastructure, illustrating the complex interactions between tasks, resources, and logistical operations. The dataset serves as both a demonstration of the computational magnitude and a detailed representation of how various task components utilize resources. The diverse characteristics, which jointly provide an overview of the cluster's activities, encompass:

Temporal Characteristics: These metrics precisely measure the time during which resource utilization was observed for a given task. The following are the identifiers used in this study: participant ID, condition ID, and trial. The identification numbers associated with job positions: Each task can be traced back to a parent job, and the use of an identification (ID) assists in establishing this connection.

The topic of discussion pertains to machine identification numbers. The utilization of this identification (ID) aids in the precise identification of the machine responsible for executing a given task, as tasks are completed on designated machines.

The Utilization of Resources: The average rate of CPU usage. When considering the entire observation period, this provides a concise representation of the CPU's activity.

The metrics related to memory usage: Canonical Memory Usage refers to the customary amount of memory occupied by a process, encompassing the page cache while excluding any outdated pages. The assigned memory usage refers to the amount of memory that is allocated for a specific job, regardless of whether it is fully utilized or not. Unmapped Page Cache refers to a cache mechanism that stores pages of data that have not been mapped to any specific memory location. This cache is typically used in systems where memory mapping is dynamic and pages can be mapped to different locations. The analysis of memory usage aids in comprehending how a job interacts with cached data.

The analysis of maximum memory utilization is crucial for comprehending intermittent spikes in work demands. The interactions between disks. The measurement of disk input/output (I/O) performance: These metrics, such as the mean disk I/O time and the maximum disk I/O time, offer valuable insights into how jobs perform data read and write operations. The metric of Mean Local Disk Space Used provides insight into the level of disk utilization associated with a given operation, hence offering valuable information for processes involving substantial datasets. Specialized metrics refer to specific quantitative measures that are tailored to assess and evaluate particular aspects or domains within a given context. The concept of the Consumer Price Index, which stands for Cycles Per Instruction, refers to the average number of clock cycles required to execute a single instruction in a CPU need to execute an instruction

is a critical indicator for performance analyzers. A high CPI may indicate the presence of bottlenecks. The metric known as Memory Accesses per Instruction (MAI) provides insight into the frequency with which a job accesses memory about its instructions. This metric serves as an indicator of processes that heavily rely on memory. The sampled CPU usage provides a detailed viewpoint by presenting the average utilization of the CPU for a randomly chosen second within the designated observation timeframe. Additional characteristics: The concept of Linux memory overheads pertains to the amount of memory that is utilized by a Linux container to carry out tasks on behalf of the user. The dataset primarily captures information regarding local disk usage, deliberately excluding large-scale systems to maintain a specific emphasis on storage interactions that are relevant to specific tasks.

When these features are analyzed as a whole, they provide a comprehensive perspective on the execution of jobs within the cluster. The wide range of capabilities available in this system allows researchers and analysts to thoroughly examine operations at various levels of detail, ranging from the overall job level to the precise relationships between resources in particular tasks.

The paper introduces new datasets derived from publicly available Google Cloud usage [49] trace data DCRUD . During the data analysis process, these datasets have been uniquely labelled for us to identify data drift, which is defined as a change in the statistical properties of data over time, which is a crucial concern for machine learning and data analysis. Specifically, criteria and metrics have been defined to label drift within the DCRUD dataset, which focuses primarily on three major resources within the respective cloud environments within which they are stored. The resources could include CPU utilization, memory consumption, and network traffic metrics. Several steps are involved in preparing labelled datasets for drift detection, including data collection, labelling, partitioning, script, and evaluating them. As Google Cloud environments evolve, the use of these datasets facilitates the development of drift detection models and tools, making it easier to monitor and manage resources.

In the context of monitoring CPU, disk I/O time, and memory usage, utilizing the established capacity groups – ''high,'' ''medium,'' and ''low'' – as benchmarks for resource utilization, the analysis involves scrutinizing the time series data over gradual and sudden drift. To effectively recognize change patterns, the continuous values indicating usage need to be segmented into capacity groups. These groupings rely on threshold values established in paper [52], which offer insights into the categorization of ''high'', ''medium'', and ''low'' usage based on these thresholds. Capacity groups of each resource are given in Table 3 & 4.

1) HEURISTIC FOR SUDDEN CHANGE DETECTION

In a complex system that has 15 distinct capacity groups, setting specific thresholds for each of these groups is an essential strategy for proactive drift detection in a complex system with 15 distinct capacity groups. To detect any significant deviation from the expected norm, organizations must closely monitor the usage or performance metrics of resources within these groups. An abrupt shift in resource demands or behaviour is indicated by a sudden jump of two or more units within a capacity group. Heuristic for Gradual Change Detection: A notable pattern to identify gradual changes is when a value transitions from one capacity group to another, with a shift of three or more groups concerning time. It is noteworthy that the initiation of the progressive change detection process can occur even with a relatively minor increase of two to three groups. The time frame of this change may vary in different settings, highlighting the importance of adaptability in acknowledging these incremental transformations. A gradual change can be inferred when there is a consistent and prolonged rise or decline in the utilization of resources across multiple time steps.

TABLE 3. CPU and memory usage groups.

Group	Range	Name
1	0.0000 - 0.0345	Minimal Resource Load
2	0.0345 - 0.0517	Light Resource Utilization
3	0.0517 - 0.0689	Low Resource Demand
4	0.0689 - 0.0759	Very Low Usage
5	0.0759 - 0.0776	Extremely Low Utilization
6	0.0776 - 0.1200	Negligible Demand
7	0.1200 - 0.1362	Moderate Resource Load
8	0.1362 - 0.1505	Balanced Resource Usage
9	0.1505 - 0.1638	Intermediate Resource Demand
10	0.1638 - 0.1800	Medium Resource Utilization
11	0.1800 - 0.6000	Fairly High Demand
12	0.6000 - 0.8909	Intensive Resource Load
13	0.8909 - 0.9679	Heavy Resource Utilization
14	0.9679 - 0.9973	High Resource Demand
15	0.9973 - 1.0000	Extremely High Utilization

TABLE 4. Disk IO time capacity groups.

Group	Range	Name
1	0.000 - 0.034	Low Resource Demand
2	0.080 - 0.120	Balanced Resource Usage
7	0.120 - 0.300	Medium Resource Utilization
8	0.300 - 0.600	Heavy Resource Utilization
14	0.600 - 0.900	High Resource Demand
15	0.900 - 1.000	Extremely High Utilization

To provide a systematic methodology for handling cloud usage trace data, the suggested comprehensive model has been dissected into its constituent elements, which are briefly outlined in Figure 2. The core functionality of the system relies on the process of annotating data for drift detection, as well as the utilization of advanced model methodologies and drift detection. The model's ability to effectively handle non-Gaussian data distributions and accurately represent drift across multiple dimensions contributes to its credibility.

C. COLLECTION AND ANALYSIS OF USAGE TRACES

The study model begins by gathering and surveillance cloud usage trace data, as mentioned in section A. This stage encompasses the collection and comprehensive examination

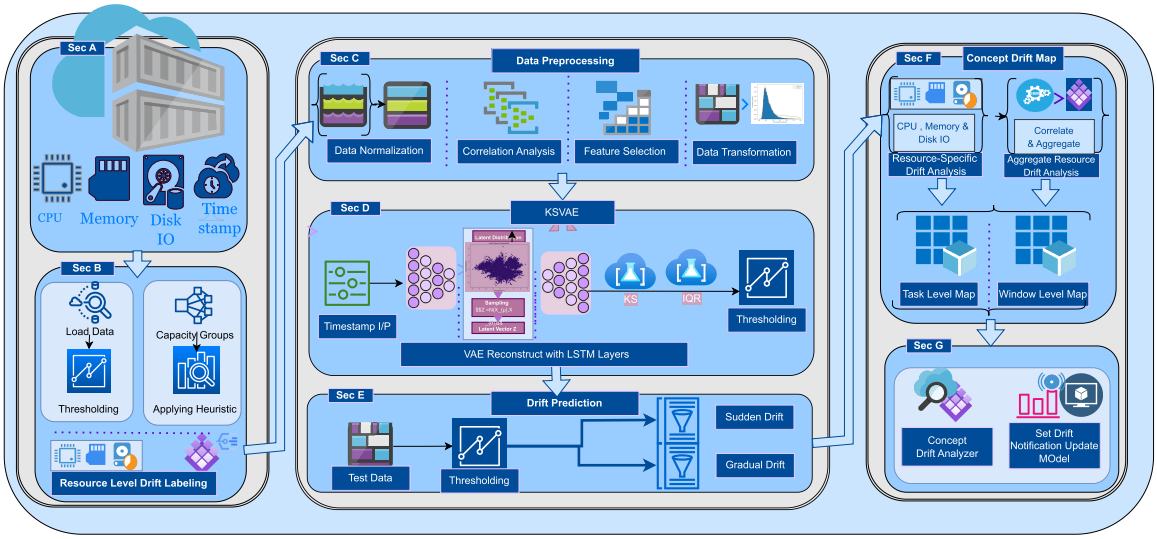

FIGURE 2. DRIFTNET-EnVACK: Drift Recognition and Identification with the Ensemble of Variational Auto-encoder featuring Sequential Contextual Network Extended Kolmogorov-Smirnov Test ; Concept drift map for gradual and sudden drift at Task vs Window level; Drift warning to detection.

of usage trace data originating from a cloud cluster. The usage trace data comprises essential indicators related to the utilization of resources, such as CPU, Memory, and Disk I/O, among others. This particular component functions as the foundational basis for following analytical procedures, offering crucial insights into the distribution of resources and the patterns of performance inside the cloud architecture. Later in section B of the study, the focus shifted towards the process of dataset annotation. This phase involves the meticulous annotation of both gradual and sudden changes within the dataset. The objective is to systematically document instances in which the consumption of resources exhibits discernible shifts or fluctuations. Accurate data classification plays a vital role in efficiently detecting and examining anomalous patterns, hence facilitating the identification of potential performance disparities within the cloud infrastructure.

The process of data pre-processing is a crucial step in preparing the dataset for thorough analysis and training of models. This stage involves a variety of complex pre-processing steps:

1) Data Normalization: The process of data normalization involves carefully scaling all data values to fit inside a specified range of 0 to 1. The process of normalization guarantees uniform scaling across all features, therefore promoting optimal model convergence and the creation of accurate predictions. IQR (Interquartile Range) outlier detection to lessen the effect.
2) Feature Selection: Stringent feature selection methodologies are consistently employed to find and keep solely the most relevant features. The primary objective of this process is to delete features that are either irrelevant or redundant, hence resulting in a reduction in dimensionality and computing complexity.
3) Feature Transformation: After the process of feature selection, the selected features are subjected to a power transformation. This modification plays a crucial role in mitigating skewness, a prevalent attribute observed in the distributions of cloud data. The mitigation of skewness is of utmost importance, as it can have adverse impacts on both the performance and interpretability of models.

D. ENSEMBLE OF VARIATIONAL AUTO-ENCODER FEATURING SEQUENTIAL CONTEXTUAL NETWORK EXTENDED KOLMOGOROV-SMIRNOV TEST (ENVACK)

The proposed model architecture named DRIFTNET-EnVACK: Drift Recognition and Identification with the Ensemble of Variational Auto-encoder featuring Sequential Contextual Network Extended Kolmogorov-Smirnov Test is presented in section D. It consists of a combination of a VAE featuring a stack of sequential contextual networks. Later loss function and thresholding are defined in this section. The DRIFTNET-EnVACK algorithm 1 showcases the presented model architecture.

1) DRIFTNET-EnVACK

DRIFTNET-EnVACK is a Ensemble of Variational Auto-encoder featuring Sequential Contextual Network Extended Kolmogorov-Smirnov Test (EnVACK) . Our contribution combines the effectiveness of Variational Auto-encoders (VAEs) for distinguishing characteristics to acquire probabilistic mappings in the latent space with the novel sequential contextual layers to record and store knowledge about progressive changes in the data shown in Figure 3. This two-pronged strategy improves the model's capacity to produce precise and context-sensitive representations of the observed data, making it appropriate for drift detection of complex cloud traces where both probabilistic modeling and sequential information are essential.

Encoder reduce dimensions allows to capture important characteristic later followed by decoding to reconstruct the original data to compare changes. The encoder

network primarily converts input data into the following vectors: mean ($\boldsymbol{\mu}$) and standard deviation ($\boldsymbol{\sigma}$). The vectors above are used to build a latent space probability distribution. Equation (1, 2) represents the encoding process with Sequential Contextual Network (SCN), transforming input data through equation (5) into vector $\boldsymbol{\mu}$. In equation (3,4), the function "Encoder" encodes input data using equation (6) to produce the output symbol $\boldsymbol{\sigma}$.

$$\text{Encoder Output}_1 = \text{SCN}_1(\text{Input Data}) \tag{1}$$

$$\vdots$$

$$\text{Encoder Output}_n = \text{SCN}_n(\text{Encoder Output}_{n-1}) \tag{2}$$

$$\text{Decoder Output}_1 = \text{SCN}_1(\text{ Latent Space Representation}) \tag{3}$$

$$\vdots$$

$$\text{Decoder Output}_m = \text{SCN}_m(\text{Decoder Output}_{m-1}) \tag{4}$$

$$\boldsymbol{\mu} = \text{Encoder(Input Data)} \tag{5}$$

$$\boldsymbol{\sigma} = \text{Encoder(Input Data)} \tag{6}$$

The process of sampling from the latent space. The generation of latent space samples is achieved through reparameterization. The sampling process involves utilizing a basic normal distribution, namely the $\mathcal{N}(0, 1)$ distribution with a mean of 0 and a standard deviation of 1. This distribution is then modified by incorporating parameters $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ calculated using equation (5,6). The equation (7) produces a sample from the latent space by adding the mean vector ($\boldsymbol{\mu}$) to the element-wise multiplication of the standard deviation vector ($\boldsymbol{\sigma}$) and random noise vector ($\boldsymbol{\epsilon}$).

$$\text{Latent Space Sample} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon} \tag{7}$$

Let $\boldsymbol{\epsilon}$ denote a random sample from a normal distribution, $\mathcal{N}(0, 1)$. The decoder algorithm restores encrypted data. The decoder network is often used to translate latent samples into data spaces. Recovery of the original input is its main purpose. Equation (8) uses the decoder function on a latent space sample to reconstruct the output.

$$\text{Reconstructed Output} = \text{Decoder(Latent Space Sample)} \tag{8}$$

2) LOSS FUNCTION

The loss calculation is a crucial component in machine learning algorithms. The loss function consists of two components:

a: RECONSTRUCTION LOSS

The reconstruction loss refers to the discrepancy between the input and output generated by a model during the process of reconstruction. The faithfulness of the decoded samples to the original inputs is commonly assessed by measuring their similarity using a suitable Trimmed mean squared error (MSE) distance metric. The reconstruction loss is defined as the MSE of input data with the reconstructed output in equation (9).

$$\text{Reconstruction Loss} = \text{MSE(Input Data, Reconstructed Output)} \tag{9}$$

b: KL DIVERGENCE LOSS

The KL Divergence Loss is utilized to regularize the encoded vectors to align them with a conventional normal distribution.

$$\text{KL Divergence Loss} = -\frac{1}{2}\sum_{i=1}^{n}\left(1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2\right) \tag{10}$$

The equation (10) represents the KL Divergence Loss, which is calculated as the negative half of the sum from i = 1 to n of the expression $(1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2)$. The VAE produces encoded vectors with the following characteristics as a result of these combined effects within the KL divergence loss equation:

- The variance (σ_i^2), mean (μ_i), and the number of components (n) represents in the multivariate distribution.
- μ_i closer to zero: minimizing the overall loss function during training pushes the VAE to adjust the encoder network to produce means closer to zero.
- σ_i^2 variance closer to one: minimizing the loss function encourages the VAE to adjust the encoder to produce variances closer to one, promoting a well-spread latent space suitable for capturing variations in the data.
- $\log(\sigma_i^2)$: The natural logarithm function of variance to prevent the collapse of the latent space, it is important to avoid positive variances.

E. THRESHOLD ESTIMATION

Section E elucidates our innovative approach to threshold estimation, which employs statistical techniques to adaptively determine drift thresholds. This departure from static thresholds permits dynamic adjustments based on data distribution characteristics, rendering drift detection more precise and robust. For non-Gaussian data distributions, such as those encountered in cloud usage trace data, the KS method proves effective in calculating the threshold, enhancing the model's capacity to detect drift patterns accurately.

The KS test compares two windows' distributions. That determines how well a dataset matches a distribution like the normal distribution. The Kolmogorov-Smirnov (KS) test can assess drift value distribution and threshold selection in our model. Many academic fields use the Kolmogorov-Smirnov (KS) test to determine thresholds [53], [54]. The empirical cumulative distribution function (ECDF) of actual drift values is compared to the CDF of a theoretical distribution. The highest vertical divergence between cumulative distribution functions is the KS statistic. The determination of the threshold is thereafter established by selecting a significance level (α) for the Kolmogorov-Smirnov test. The KS statistic, denoted as (11), is defined as the maximum absolute

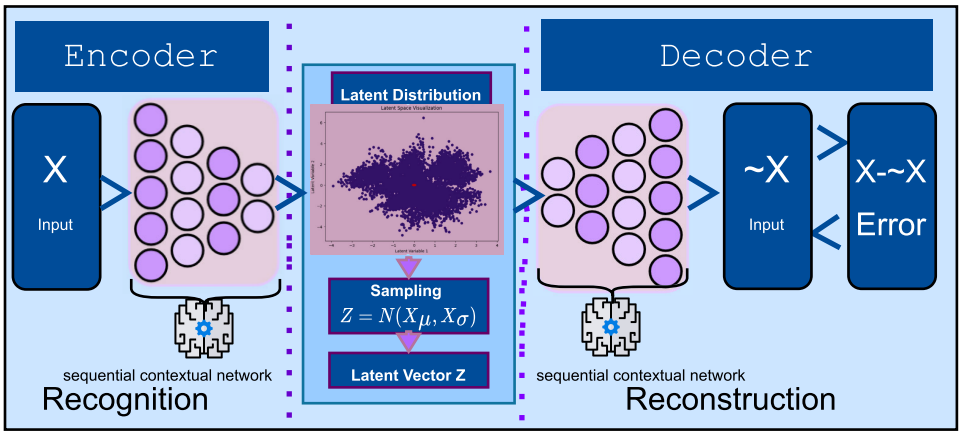

FIGURE 3. Elucidating the underlying mechanics of variational auto-encoders enhanced with a contextual network for deeper understanding and insights: Uncovering latent space's richness as a multidimensional representation of the underlying non-gaussian probability distribution with.

difference that is observed between ECDF c, $F_{\text{observed}}(x)$, and the CDF, $F_{\text{theoretical}}(x)$. The variable x is used to denote the drift value.

$$\text{KS Statistic} = \max |F_{\text{observed}}(x) - F_{\text{theoretical}}(x)| \tag{11}$$

where: - $F_{\text{observed}}(x)$ is the ECDF of observed drift values. - $F_{\text{theoretical}}(x)$ is the CDF of the theoretical distribution. - x represents the drift value. The threshold value can be calculated using equation (12), which is given by the formula below. The symbol α represents the significance level, which is commonly denoted as 0.05 to indicate a 5 % significance level. The variable n represents the sample size of drift values.

$$\text{Threshold} = \sqrt{-\frac{1}{2}\ln\left(\frac{\alpha}{2}\right)\frac{1}{n}} \tag{12}$$

where: - α is the significance level (e.g., 0.05 for a 5 - n is the sample size of drift values.

F. CONCEPT DRIFT MAP

A concept drift map for non-Guassian distribution is introduced in section F of Figure 2 zoomed in Figure 4, which offers nuanced insights into drift detection across multiple levels and distinct resources. This map is instrumental in capturing drift at both the task and window levels, as well as for individual and combined resources. It enables localized and global drift detection, facilitating targeted analysis and responses to evolving data patterns or model behaviour. Concept drift detection is performed at the task level by monitoring model performance on individual data instances and at the window level by tracking performance over time using a sliding window approach. Moreover, the model scrutinizes individual resources separately and in conjunction, yielding a comprehensive understanding of resource-specific and collective drift phenomena. The model relies on non-normal distribution modelling to discern and monitor CPU and memory usage patterns covered in last section G. By accommodating the non-Gaussian distribution characteristics of these variables, the model endeavours to enhance the detection and comprehension of CPU and memory usage dynamics ultimately improving optimum model updations.

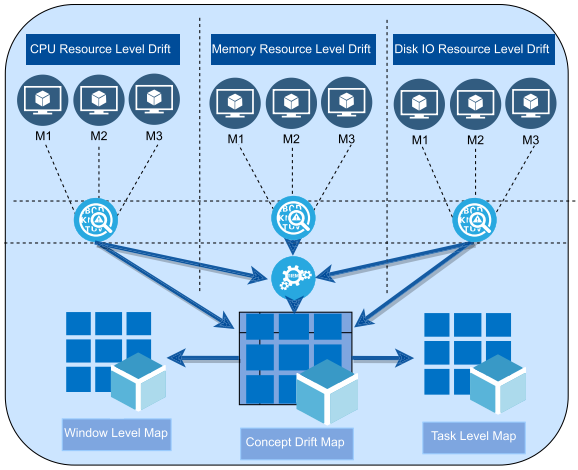

FIGURE 4. Our novel approach to concept drift map: An extensive framework designed to discover and visualize subtle conceptual shifts in resource utilization, both at the individual and aggregated levels, across various tasks and time windows.

G. ENVACK ANALYSIS

In Figure 5, mentions how EnVACK is working as a proactive approach by offline monitoring the data distribution and response only when drift is occurred. Traditional models are frequently retrained to maintain their performance, especially when data shifts occur. However, this process can be costly and requires a lot of processing power. Fortunately, there is a more affordable and efficient solution, which is EnVACK drift detection features to monitor the model's stability continuously. As data distribution drifts, reconstruction errors typically increase, indicating potential problems. Early detection of drift eliminates the need for repeated retraining, making it a more cost-effective and proactive approach that reduces the likelihood of incorrect predictions caused by outdated models. Despite the fact that drift detection also requires processing power, it is generally less expensive than repeated retraining, particularly in the case of infrequent

drifts. Another aspect is to be robust to noise and outliers. There is no doubt that VAEs can effectively deal with noise in your data, however, special handling may be required for outliers and they are not designed for handling data drift without retraining. In the code, trimmed mean squared errors are added in order to account for noise and outliers. To reduce the effect of outliers, the data is pre-processed with normalization and IQR (Interquartile Range) outlier detection before being sent to the EnVACK for a more accurate analysis.

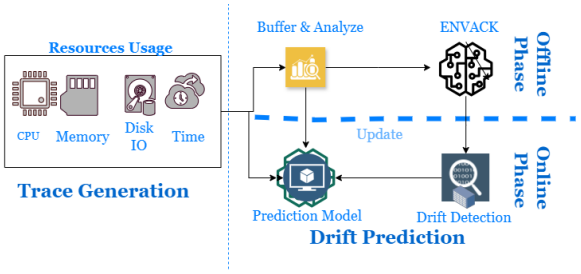

FIGURE 5. Strategic placement of EnVack as a drift monitor.

IV. RESULTS AND DISCUSSION

A. EXPERIMENTAL SETUP

The experimental setup specifies the tools and resources used in the research process, including the hardware and software infrastructure for data processing, modeling, and analysis. Python was the main programming language used in our experiment, and we used Google Colab because of its effective GPU runtime to speed up the model training process. We developed our model implementation by utilizing the TensorFlow library, making use of its high-level Keras API for building and training neural networks. By using TensorFlow's rich feature set and Colab's processing power, this method allowed us to construct and deploy our models more quickly. The combination of TensorFlow, Colab, and Python allowed for quick experimentation and iteration cycles, which improved the productivity and efficacy of our research projects. Building robust machine learning models requires dividing your dataset. The model is trained on a sizable training set (60% of the total data), prevented from overfitting by a validation set (20%) and evaluated on a testing set (20%). As a general guideline, the 60:20:20 split provides a balance between training the model and having an adequate amount of data for testing and validation.

This particular design of the neural network produced the best results for this experiment are summarised in Table 5. There are two processing stages between the input and the output because the network has two layers of depth. There are 64 and 32 neurons in each of these layers, respectively. The network learns a hidden representation called latent space, which has 20 dimensions. Training took place over a 23000-iteration period with a 32-batch size, processing 32 data points at once. The complexity of setting up and running this network is then examined in Table 5. The network depth, the number of training iterations, and the batch size all affect how long the training process takes. This indicates that when the network becomes deeper, more iterations become necessary, or the batch size increases the training time increases. The depth and the quantity of neurons in each layer determine the space complexity for training. Put another way, deeper networks with more neurons need more storage. The use of the trained network, or deployment complexity, is less difficult. Like training space complexity, the time complexity for deployment increases with depth and neuron count. On the other hand, depth has no bearing on the space complexity for deployment–only the quantity of neurons does. This means that the network needs less storage for prediction after training than it did before.

TABLE 5. Hyper parameters and complexities of EnvAck model.

Parameters & Complexity	Training	Deployment
Time Complexity	$O(D \times T \times N)$	$O(D \times W)$
Space Complexity	$O(D \times W)$	$O(W)$
Parameters	$O(D \times W)$ $O(W)$ (D: Depth, T: Iterations, N: Batch size, W: Layers parameters) ($D = 2, T = 23000, N = 32, W = 64$)	

B. COMPARISON

A comprehensive study on ablation was undertaken to further deep learning. The systematic investigations helped clarify the effects and contributions of framework components and techniques. These studies sought to identify model performance and resource efficiency essentials. Ablation experiments examined architectural aspects, which were divided into four groups: Auto Encoder, VAE and DRIFTNET-EnVACK . These architectural components' effects on model performance have been thoroughly studied by carefully eliminating or altering them. The project also examined feature engineering, hyperparameter optimization, and advanced deep learning approaches, focusing on attention processes beyond architectural features. A careful analysis of multiheaded attention, self-attention processes, and ensemble methods was done. By carefully adjusting and excluding variables, these ablation studies have identified the key factors that greatly impact the model's efficacy.

Ablation research has helped refine the model over time. The models described above have helped us understand the complex interaction of several elements. The above observations show great promise for the scientific community and emphasize the need for thorough deep-learning research. We have used VAEs with contextual networks along KS inference to solve drift detection's complex problem. Our data show several important benefits of this approach: First, DRIFTNET-EnVACK guided by variational inference enabled robust drift detection. Our model captured subtle shifts and nuanced patterns in data distributions across time by estimating complex posterior distributions. Our methodology's robustness, especially in high-dimensional

Algorithm 1 Ensemble of Variational Auto-Encoder featuring Sequential Contextual Network Extended Kolmogorov-Smirnov Test (EnVACK)

Require: *TrainX*, *TrainY*, *ValidationX*, *ValidationY*, *TestingX*, *TestingY*
Ensure: Variational Auto-encoder (VAE)-based Drift Detector
procedure Preprocessing
    Normalize using MinMaxScaler *TrainX* and *ValidationX*
    Apply PowerTransformer **return** *TrainX* and *ValidationX*
end procedure
procedure VAE Layer(input_shape, return_sequences = True)
    Initialize biases with zeros b **return** Weighted sum of input
end procedure
procedureVAE Model
    Encoder: Create an input layer: $Input(shape = (timesteps, input\_dim))$
    Add a stack of sequential contextual network layers:
    The output $o_i^{(l)}$ is computed as follows:

$$\text{SCN}_n(\text{Encoder Output}_{n-1}) = \phi\left(\sum_j W_{ij}^{(l)} h_j^{(l-1)} + b_i^{(l)} + \mathcal{C}(\text{Encoder Output}_{n-1})\right)$$

where ϕ is the activation function.
    Let $h_i^{(l)}$ be the output of the i-th neuron in the l-th layer of the SCN, and $\mathcal{C}$ represents the contextual formalism.
    The contextual formalism $\mathcal{C}$ incorporates relevant information to enhance the encoder output.
    Latent Space: Compute the mean and log variance of the latent space:

$$z\_mean = Dense(latent\_dim)(encoder\_output)$$
$$z\_log\_var = Dense(latent\_dim)(encoder\_output)$$

    Use the reparameterization trick to sample from the latent space:

$$z = \text{Sample from } N(z\_mean, e^{z\_log\_var})$$

    Decoder: Add a stack of sequential contextual network layers:
    The output $o_i^{(l)}$ is computed as follows:

$$\text{SCN}_n(\text{Decoder Output}_{n-1}) = \phi\left(\sum_j W_{ij}^{(l)} h_j^{(l-1)} + b_i^{(l)} + \mathcal{C}(\text{Decoder Output}_{n-1})\right)$$

where ϕ is the activation function.
    Let $h_i^{(l)}$ be the output of the i-th neuron in the l-th layer of the SCN, and $\mathcal{C}$ represents the contextual formalism.
    The contextual formalism $\mathcal{C}$ incorporates relevant information to enhance the decoder output.
    The contextual formalism $\mathcal{C}$ incorporates relevant information to enhance the encoder output.

and complicated datasets, shows its ability to recognize changing concepts. Second, DRIFTNET-EnVACK data modelling flexibility was crucial. Drift properties vary widely between applications. Our results showed how the model can be customized for drift detection cloud resource usage tasks due to the ability to deal with non-parametric and high variation in data. Our technique is more versatile and applicable across areas due to this adaptability with a bit of fine-tuning. Our methodology aids model adaption and corrective action decisions by accurate drift warnings.

DRIFTNET-EnVACK excelled at drift pattern identification beyond detection. This inference, modelled the data points far from the learned distribution. This functionality flagged data instances indicating drift or aberrant behaviour, helping identify drift early. Our research relied on VAEs' probabilistic nature to forecast probabilistically. By estimating posterior distributions, we may forecast future data points and quantify uncertainty. This predictive capability helps anticipate drift and plan accordingly.

C. ANALYSIS

The results of our investigation are convincing evidence of the potency of our novel non-parametric VAE with contextual memory technique for cloud drift identification. Notably, our model outperformed traditional VAE as shown in Table 6 and also benchmarked unsupervised VAE [40] models by a significant 5% margin in terms of f-score when compared with synthetic sine data 7 and visualized in

Algorithm 2 Ensemble of Variational Auto-Encoder featuring Sequential Contextual Network Extended Kolmogorov-Smirnov Test (EnVACK) (Continued)

22: Define L1 & L2 regularization with coefficients:

$$regularization = tf.keras.regularizers.L1L2()$$

23: Compute the regularization loss for the first trainable weight:

$$regularization\_loss = regularization(Auto - encoder.trainable\_weights[0])$$

24: **for** $weight in Auto - encoder.trainable\_weights[1:]$ **do**
25: Compute the regularization loss & add it to *regularization_loss*
26: **end for**
27: Add the regularization loss to the Auto-encoder:

$$Auto - encoder.add\_loss(lambda : regularization\_loss)$$

28: Compile

$$Auto - encoder.\text{compile}(input\_layer, decoder, u, d, adam, `binary\_crossentropy', \alpha)$$
$$model.\text{fit}(X_{\text{train}}, Y_{\text{train}}, \text{batch\_size} = b, \text{epochs} = e, \text{class\_weight} = w)$$

29: Build and train multiple VAE instances:

$$VAE = \text{Model}(inputs = input\_layer, outputs = decoder)$$
$$VAE.add\_loss(lambda : regularization\_loss)$$
$$VAE.compile(opt =' adam', loss =' binary\_crossentropy', metrics = [`accuracy'])$$

30: Create *n_modes* instances of the Auto-encoder
31: Train each VAE instance on *TrainX* and *ValidationX*
32: Predict drift based on the calculated thresholds using *ValidationY* and *TestingY*
33: Calculate threshold using KS (Kolmogorov-Smirnov) test:

$$\text{KS Test Statistic: } D = \max_x |F_1(x) - F_2(x)|$$

where $F_1(x)$ is the cumulative distribution function (CDF) of the 1st sample, and $F_2(x)$ is the CDF of the 2nd sample.

34: Calculate f-score, accuracy, precision, and recall
35: Drift detection results. =0

Figure 6. They [40] also compared their results on synthetic data and did not evaluate them on real datasets. We have also evaluated our techniques with benchmark techniques summarized in Table 8. These results are compared in terms of error. This result is particularly important since it emphasizes the usefulness of our strategy in improving the accuracy of cloud drift detection. Our model greatly surpassed the conventional AE and VAE models shown in Table 6. This significant improvement in f-score may result in concrete advantages for cloud systems, including better resource management, fewer system failures, and improved user experiences. Contextual memory along with VAE ability to map probability distribution was also incorporated into our model to increase its adaptability and resilience. It explicitly models data as a probability distribution by considering the entire distribution. It provides a more comprehensive and accurate representation of the data. They learn a lower-dimensional latent space into which data is mapped, which can have a meaningful structure. It enables the separation of underlying sources of variance in data, revealing how diverse factors affect the overall distribution. This allows them to capture the variability and uncertainty present in the data, which is often lost when using statistical methods. They are also capable of detecting data points that differ significantly from the learned distribution. This is difficult to achieve using typical statistical methods without taking into account the co-relationship in the data's intrinsic structure. They are adaptable to various data distributions and do not rely on strong distributional assumptions. Cloud datasets exhibit high-dimensional data and non-linear patterns. Its ability to capture these intricacies can lead to more accurate approximations of the underlying distribution. As, it can also handle high-dimensional data successfully, which might be difficult for typical statistical methods. Hidden structures in high-dimensional data are frequently difficult to detect using standard statistics. Because of their probabilistic and generative character, they provide a more extensive and nuanced representation of data distributions, leading to

TABLE 6. A summarized comparative analysis of drift detection architectures: Investigating the efficacy of supervised and unsupervised paradigms for adaptive learning in dynamic environments.

Model	Key Model Architecture	Features Engineering	Hyper Parameter	Attention	Accuracy	Precision	Recall	F-Score
AE	Model 1	Yes	Yes	Yes	0.70	0.73	0.72	0.72
	Model 2	No	No	No	0.65	0.68	0.67	0.68
VAE	Model 1	Yes	Yes	Yes	0.80	0.84	0.78	0.80
	Model 2	No	No	No	0.77	0.78	0.72	0.77
EnVACK	Model 1	Yes	Yes	Yes	0.85	0.94	0.80	0.85
	Model 2	No	NO	No	0.80	0.90	0.81	0.80

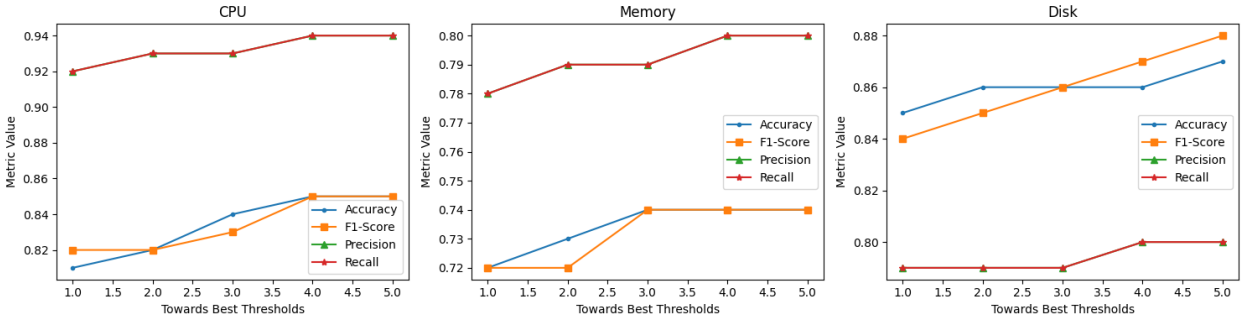

FIGURE 6. An visualised comparative analysis of EnVACK across resources.

TABLE 7. Performance evaluation of EnVACK drift detection mechanisms: Leveraging google dataset across CPU, memory, and Disk resources, and synthetic data to unveiling insights into robustness against gradual and abrupt drift patterns.

Dataset	Google			Sine	
	CPU	Memory	Disk	Abrupt	Gradual
Accuracy	0.85	0.74	0.87	0.95	0.95
Macro f-Score	0.85	0.74	0.88	0.95	0.95
Micro f-Score	0.85	0.74	0.80	0.96	0.95
Recall	0.80	0.70	0.98	0.90	0.90
Precision	0.94	0.80	0.80	0.90	0.90

TABLE 8. Comparing EnVACK loss to benchmark drift detectors' average error: Implications for adaptive learning in dynamic environments.

DataSet	DDM	EDDM	ECDD EWMA	STEPD	Adwin	HDDMA	HDDMW	EnVACK
Sine	0.016	0.047	0.031	0.0312	0.0156	0.251	0.282	0.006
Google	0.007	0.006	0.007	0.006	0.008	0.008	0.008	0.005

more accurate approximations and superior insights. This characteristic made it possible for our method to adapt to changing cloud conditions, which is essential in dynamic cloud settings. Furthermore, our model's non-parametric components provide a clear benefit when managing intricate, non-Gaussian distributions that are frequently present in real-world cloud drift data summarized in Table 7. Our method stands out from the rest and offers a viable way to tackle cloud drift detection problems in the future by skillfully capturing the subtleties of cloud drift patterns.

Our findings provide important insights into the field of cloud drift detection. To begin, the significant accuracy improvement attained by our approach emphasizes its practical value. It has the potential to improve the dependability and performance of cloud-based systems, making them more resistant to disruptions caused by drift. The scalability and durability of our paradigm, enabled by the addition of contextual memory, are especially important in the continual gradual drift in cloud computing. Because of this versatility, our model can efficiently accommodate changes in cloud drift patterns, making it a valuable tool for system managers and cloud service providers. The non-parametric aspect of our model is also noteworthy since it allows the model to manage the complex and dynamic distributions inherent in cloud systems. This novel element distinguishes our technique from previous methods and highlights its potential for tackling future issues in the field of drift detection in cloud systems. Finally, with its remarkable performance improvements, adaptability, and capacity to manage intricate data distributions, our DRIFTNET-EnVACK: Drift Recognition and Identification with the Ensemble of Variational Autoencoder featuring Sequential Contextual Network Extended Kolmogorov-Smirnov Test represents a significant advancement in the realm of cloud drift detection, making it a valuable contribution to the broader field of drift detection in cloud systems.

V. CONCLUSION

In this paper, we introduced our unsupervised VAE with contextual memory enhanced with a statistic concept drift detection method named DRIFTNET-EnVACK . The results are shown on the artificial and cloud datasets that our

method was able to detect concept drifts using activity probability distribution derived from normal train data. Moreover, we showed that our new approach is more accurate than statistical and deep learning models. We evaluated our method on a real world dataset showing an improvement of 5% f-score. Our contribution is also a novel approach to the concept drift map an extensive framework designed to discover and visualize subtle conceptual shifts in resource utilization both at the individual and aggregated levels across various tasks and time windows. Furthermore, we presented the actual implications of our research, emphasizing its potential for load balancing with a specific emphasis on consumption trace. Our models' flexibility to a wide range of domains is evidenced by their suppleness in dealing with non-parametric approaches. We can further enhance the results by considering non-paramteric approaches in VAE latent space. In closing, our study offers a substantial advancement in the field of resource utilization prediction. We feel that the findings and approaches given in this study lay the platform for future research, and we anticipate that additional advances and breakthroughs in cloud computing will build on this foundation.

REFERENCES

[1] M. Zaharia, B. Hindman, A. Konwinski, A. Ghodsi, A. D. Joseph, R. H. Katz, S. Shenker, and I. Stoica, "The datacenter needs an operating system," in *Proc. 3rd USENIX Workshop Hot Topics Cloud Comput.*, 2011, pp. 1–5.

[2] M. Jangjou and M. K. Sohrabi, "A comprehensive survey on security challenges in different network layers in cloud computing," *Arch. Comput. Methods Eng.*, vol. 29, no. 6, pp. 3587–3608, Oct. 2022.

[3] M. Saraswat and R. C. Tripathi, "Cloud computing: Comparison and analysis of cloud service providers-AWs, Microsoft and Google," in *Proc. 9th Int. Conf. Syst. Model. Advancement Res. Trends (SMART)*, Dec. 2020, pp. 281–285.

[4] D. Puthal, B. P. S. Sahoo, S. Mishra, and S. Swain, "Cloud computing features, issues, and challenges: A big picture," in *Proc. Int. Conf. Comput. Intell. Netw.*, Jan. 2015, pp. 116–123.

[5] A. Sunyaev, *Internet Computing: Principles of Distributed Systems and Emerging Internet-Based Technologies*. Cham, Switzerland: Springer, 2020, pp. 195–236.

[6] C. T. Kamanga, E. Bugingo, S. N. Badibanga, and E. M. Mukendi, "A multi-criteria decision making heuristic for workflow scheduling in cloud computing environment," *J. Supercomput.*, vol. 79, no. 1, pp. 243–264, Jan. 2023.

[7] Y. Mansouri, V. Prokhorenko, F. Ullah, and M. A. Babar, "Resource utilization of distributed databases in edge–cloud environment," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9423–9437, Jun. 2023.

[8] S. Bharany, S. Sharma, O. I. Khalaf, G. M. Abdulsahib, A. S. Al Humaimeedy, T. H. H. Aldhyani, M. Maashi, and H. Alkahtani, "A systematic survey on energy-efficient techniques in sustainable cloud computing," *Sustainability*, vol. 14, no. 10, p. 6256, May 2022.

[9] A. Rahimikhanghah, M. Tajkey, B. Rezazadeh, and A. M. Rahmani, "Resource scheduling methods in cloud and fog computing environments: A systematic literature review," *Cluster Comput.*, vol. 25, no. 2, pp. 911–945, Apr. 2022.

[10] L. A. Barroso and U. Hölzle, "The case for energy-proportional computing," *Computer*, vol. 40, no. 12, pp. 33–37, Dec. 2007.

[11] C. Reiss and A. Tumanov, "Heterogeneity and dynamicity of clouds at scale: Google trace analysis," in *Proc. ACM Symp. Cloud Comput.*, Nov. 2021, pp. 1–13.

[12] Y. Chen, T. Zhou, J. Wu, H. Qiao, X. Lin, L. Fang, and Q. Dai, "Photonic unsupervised learning variational autoencoder for high-throughput and low-latency image transmission," *Sci. Adv.*, vol. 9, no. 7, Feb. 2023, Art. no. eadf8437.

[13] Y. Qiu, M. S. O'Connor, M. Xue, B. Liu, and X. Huang, "An efficient path classification algorithm based on variational autoencoder to identify metastable path channels for complex conformational changes," *J. Chem. Theory Comput.*, vol. 19, no. 14, pp. 4728–4742, Jul. 2023.

[14] X. Yan, D. She, and Y. Xu, "Deep order-wavelet convolutional variational autoencoder for fault identification of rolling bearing under fluctuating speed conditions," *Exp. Syst. Appl.*, vol. 216, Apr. 2023, Art. no. 119479.

[15] R. Zemouri, R. Ibrahim, and A. Tahan, "Hydrogenerator early fault detection: Sparse dictionary learning jointly with the variational autoencoder," *Eng. Appl. Artif. Intell.*, vol. 120, Apr. 2023, Art. no. 105859.

[16] D. H. Thai, X. Fei, M. T. Le, A. Züfle, and K. Wessels, "Riesz-quincunx-UNet variational autoencoder for unsupervised satellite image denoising," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, doi: 10.1109/TGRS.2023.3291309.

[17] Y. Xia, C. Chen, M. Shu, and R. Liu, "A denoising method of ECG signal based on variational autoencoder and masked convolution," *J. Electrocardiol.*, vol. 80, pp. 81–90, Sep. 2023.

[18] E. M. Coraça, J. V. Ferreira, and E. G. O. Nóbrega, "An unsupervised structural health monitoring framework based on variational autoencoders and hidden Markov models," *Rel. Eng. Syst. Saf.*, vol. 231, Mar. 2023, Art. no. 109025.

[19] J. Tarquino and E. Romero, "Unsupervised white blood cell characterization in the latent space of a variational autoencoder," *Proc. SPIE*, vol. 12567, Mar. 2023, Art. no. 1256702.

[20] R. Bai, R. Huang, Y. Qin, Y. Chen, and C. Lin, "HVAE: A deep generative model via hierarchical variational auto-encoder for multi-view document modeling," *Inf. Sci.*, vol. 623, pp. 40–55, Apr. 2023.

[21] T. Pakornchote, N. Choomphon-anomakhun, S. Arrerut, C. Atthapak, S. Khamkaeo, T. Chotibut, and T. Bovornratanaraks, "Diffusion probabilistic models enhance variational autoencoder for crystal structure generative modeling," 2023, *arXiv:2308.02165*.

[22] F. Ye and A. G. Bors, "Continual variational autoencoder via continual generative knowledge distillation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, 2023, pp. 10918–10926.

[23] R. Pears, S. Sakthithasan, and Y. S. Koh, "Detecting concept change in dynamic data streams: A sequential approach based on reservoir sampling," *Mach. Learn.*, vol. 97, no. 3, pp. 259–293, Dec. 2014.

[24] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.

[25] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," *Knowl.-Based Syst.*, vol. 245, Jun. 2022, Art. no. 108632.

[26] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, Apr. 2007, pp. 443–448.

[27] D. T. J. Huang, Y. S. Koh, G. Dobbie, and R. Pears, "Detecting volatility shift in data streams," in *Proc. IEEE Int. Conf. Data Mining*, Dec. 2014, pp. 863–868.

[28] I. Frías-Blanco, J. D. Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 3, pp. 810–823, Mar. 2015.

[29] I. Goldenberg and G. I. Webb, "Survey of distance measures for quantifying concept drift and shift in numeric data," *Knowl. Inf. Syst.*, vol. 60, no. 2, pp. 591–615, Aug. 2019.

[30] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence—SBIA 2004*, A. L. C. Bazzan and S. Labidi, Eds. Berlin, Germany: Springer, 2004, pp. 286–295.

[31] M. Baena-Garcia, J. D. Campo-Avila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno, "Early drift detection method," in *Proc. 4th Int. Workshop Knowl. Discovery Data Streams*, vol. 6, 2006, pp. 77–86.

[32] K. Nishida and K. Yamauchi, "Detecting concept drift using statistical testing," in *Proc. Int. Conf. Discovery Sci.*, 2007, pp. 264–269.

[33] G. J. Ross, N. M. Adams, D. K. Tasoulis, and D. J. Hand, "Exponentially weighted moving average charts for detecting concept drift," *Pattern Recognit. Lett.*, vol. 33, no. 2, pp. 191–198, Jan. 2012.

[34] R. S. M. Barros, D. R. L. Cabral, P. M. Gonçalves, and S. G. T. C. Santos, "RDDM: Reactive drift detection method," *Exp. Syst. Appl.*, vol. 90, pp. 344–355, Dec. 2017.

[35] Z. Liu, C. K. Loo, and M. Seera, "Meta-cognitive recurrent recursive kernel OS-ELM for concept drift handling," *Appl. Soft Comput.*, vol. 75, pp. 494–507, Feb. 2019.

[36] D. T. J. Huang, Y. S. Koh, G. Dobbie, and A. Bifet, "Drift detection using stream volatility," in *Machine Learning and Knowledge Discovery in Databases*, A. Appice, P. P. Rodrigues, V. S. Costa, C. Soares, J. Gama, and A. Jorge, Eds. Switzerland: Springer, 2015, pp. 417–432.
[37] J. L. Lobo, J. Del Ser, I. Lana, M. N. Bilbao, and N. Kasabov, "Drift detection over non-stationary data streams using evolving spiking neural networks," in *Intelligent Distributed Computing XII*, J. D. Ser, E. Osaba, M. N. Bilbao, J. J. Sanchez-Medina, M. Vecchio, and X.-S. Yang, Eds. Cham, Switzerland: Springer, 2018, pp. 82–94.
[38] A. Seeliger, T. Nolle, and M. Mühlhäuser, "Detecting concept drift in processes using graph metrics on process graphs," in *Proc. 9th Conf. Subject-Oriented Bus. Process Manag.*, Mar. 2017, pp. 1–10.
[39] D. Zambon, C. Alippi, and L. Livi, "Concept drift and anomaly detection in graph streams," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 11, pp. 5592–5605, Nov. 2018.
[40] B. Friedrich, T. Sawabe, and A. Hein, "Unsupervised statistical concept drift detection for behaviour abnormality detection," *Int. J. Speech Technol.*, vol. 53, no. 3, pp. 2527–2537, Feb. 2023.
[41] Y. Xu and K. Wilson, "Early alert systems during a pandemic: A simulation study on the impact of concept drift," in *Proc. 11th Int. Learn. Analytics Knowl. Conf.*, Apr. 2021, pp. 504–510.
[42] G. I. Webb, L. Kuan Lee, F. Petitjean, and B. Goethals, "Understanding concept drift," 2017, *arXiv:1704.00362*.
[43] H. Yu and G. I. Webb, "Adaptive online extreme learning machine by regulating forgetting factor by concept drift map," *Neurocomputing*, vol. 343, pp. 141–153, May 2019.
[44] Z. Yang, S. Al-Dahidi, P. Baraldi, E. Zio, and L. Montelatici, "A novel concept drift detection method for incremental learning in nonstationary environments," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 1, pp. 309–320, Jan. 2020.
[45] S. Xu and J. Wang, "Dynamic extreme learning machine for data stream classification," *Neurocomputing*, vol. 238, pp. 433–449, May 2017.
[46] B. Mirza and Z. Lin, "Meta-cognitive online sequential extreme learning machine for imbalanced and concept-drifting data classification," *Neural Netw.*, vol. 80, pp. 79–94, Aug. 2016.
[47] M. Cai and Y. Li, "Out-of-distribution detection via frequency-regularized generative models," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2023, pp. 5510–5519.
[48] T. Mehmood, S. Latif, N. S. M. Jamail, A. Malik, and R. Latif, "LSTMDD: An optimized LSTM-based drift detector for concept drift in dynamic cloud computing," *PeerJ Comput. Sci.*, vol. 10, p. e1827, Jan. 2024.
[49] Google, *Google Cluster Workload Traces (2019)*, Data Set, Google Research, 2019. [Online]. Available: https://research.google/resources/datasets/google-cluster-workload-traces-2019/
[50] J. L. Lobo, "Synthetic datasets for concept drift detection purposes," Dataverse Harvard, 2020, doi: 10.7910/DVN/5OWRGB.
[51] A. Pesaranghader, H. L. Viktor, and E. Paquet, "McDiarmid drift detection methods for evolving data streams," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2018, pp. 1–9.
[52] A. K. Mishra, J. L. Hellerstein, W. Cirne, and C. R. Das, "Towards characterizing cloud backend workloads: Insights from Google compute clusters," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 37, no. 4, pp. 34–41, Mar. 2010.
[53] R. Vishwakarma and A. Rezaei, "Risk-aware and explainable framework for ensuring guaranteed coverage in evolving hardware trojan detection," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
[54] P. Sobolewski and M. Wozniak, "Comparable study of statistical tests for virtual concept drift detection," in *Proc. 8th Int. Conf. Comput. Recognit. Syst. (CORES)*, B. Burduk, K. Jackowski, M. Kurzynski, M. Wozniak, and A. Zolnierek, Eds. Heidelberg, Germany: Springer, 2013, pp. 329–337.

TAJWAR MEHMOOD received the bachelor's degree in software engineering from Fatima Jinnah Women University, Pakistan, in 2013, and the M.S. degree in software engineering from RIPHAH International University, Pakistan. She is currently pursuing the Ph.D. degree in computer science with NUST, Pakistan. From 2017 to 2022, she was a Lecturer with RIPHAH International University. She is a Lecturer with FAST NUCES University, Pakistan. Additionally, she is a Research Member with China-Pakistan Intelligent Systems (CPInS) Laboratory, SEECS-NUST, Pakistan.

SEEMAB LATIF (Senior Member, IEEE) received the Ph.D. degree from The University of Manchester, U.K. She is currently an Associate Professor and a Researcher with the National University of Sciences and Technology (NUST), Pakistan. Her research interests include artificial intelligence, machine learning, data mining, and NLP. Her professional services include an industry consultations, the conference chair, a technical program committee member, and a reviewer for several international journals and conferences. In the last three years, she has established research collaborations with national and international universities and institutes. She has also secured grants from Asian Development Bank, HEDP, World Bank, Higher Education Commission Pakistan, National ICT, HEC Technology Development Fund, and U.K. ILM Ideas. She received the School Best Teacher Award, in 2016, and the University Best Innovator Award, in 2020. She is also a Founder of NUST spin-off company, Aawaz AI Tech.

RABIA LATIF received the bachelor's degree in computer science from COMSATS Institute of Information Technology, Islamabad, and the master's degree in information security and the Ph.D. degree in cloud-assisted wireless body area networks from the National University of Sciences and Technology (NUST), Pakistan. She is currently an Associate Professor with the College of Computer and Information Sciences, Prince Sultan University, Riyadh, Saudi Arabia. Her research interests include wireless body area networks, cloud computing, information security, and artificial intelligence.

HAMMAD MAJEED is currently a Professor with the FAST National University of Computer and Emerging Sciences. His research interests include artificial intelligence, computational intelligence, machine learning, data mining and knowledge discovery, evolutionary gaming, machine vision, and robotics. He is actively involved in research in these areas.

ASAD WAQAR MALIK received the Ph.D. degree in software engineering from NUST, Pakistan. He is currently an Assistant Professor with the School of Electrical Engineering and Computer Science, NUST. His research interests include parallel and distributed simulation, cloud computing, and large-scale networks.